PYTHON DSA PRACTICALS

Complete Practical Write-up

14 Practicals | Aim • Theory • Algorithm • Time Complexity • Space Complexity • Conclusion

Subject	Data Structures and Algorithms
Programming Language	Python
Contents	14 practical write-ups in folder sequence

TABLE OF CONTENTS

01. Library Borrowing Records Using Lists

02. Linear Search and Binary Search on Customer IDs

03. Selection Sort on Employee Salaries

04. Bubble Sort and Top Five Highest Salaries

05. Undo and Redo Operations Using Stacks

06. Event Processing System Using Queue

07. Call Center Call Handling Using Queue

08. Student Record Management Using Singly Linked List

09. Expression Tree from Prefix Expression

10. Pizza Delivery Using Weighted Graph and Dijkstra's Algorithm

11. City Management Using Binary Search Tree

12. Binary Search Tree Operations

13. Hash Table Using Division Method and Linear Probing

14. Graph Traversal Using BFS and DFS

Note: Complexity is expressed in terms of n (number of elements), V (vertices), E (edges), h (tree height), and m (hash-table size), as applicable.

Practical 1

Library Borrowing Records Using Lists

Aim

To write a Python program that stores the number of books borrowed by library members and calculates the average, highest, lowest, zero-borrower count, and mode of the borrowing data.

Theory

A list is a linear data structure that stores multiple values in an ordered sequence. Each borrowing count is stored as an element of a list. The program traverses the list to calculate the total, maximum, minimum, and number of zero values. The average is obtained by dividing the total by the number of members. The mode is the value that occurs most frequently; it can be found by counting occurrences of each distinct value.

Algorithm

1. Start the program and read the borrowing counts into a list.
2. Initialize total and zero_count to 0; initialize highest and lowest using the first list element.
3. Traverse the list and update total, highest, lowest, and zero_count.
4. Calculate average = total / number of members.
5. Check the frequency of each value and identify the value or values with the greatest frequency as the mode.
6. Display the average, highest, lowest, zero-borrower count, and mode.
7. Stop.

Time Complexity

$O(n^2)$ for the simple nested-loop mode calculation; the other calculations take O(n).

Space Complexity

O(n) for storing the n borrowing counts. The basic calculations use O(1) extra space, excluding any list used to store mode values.

Conclusion

The practical demonstrates library borrowing records using lists using Python and the relevant data structure or algorithm.

Note: The average is defined only when the list contains at least one member. If several values share the highest frequency, all of them are modes.

Practical 2

Linear Search and Binary Search on Customer IDs

Aim

To implement linear search and binary search in Python to find a particular customer ID in a collection.

Theory

Searching is the process of locating a required element in a collection. Linear search checks elements one by one and works on both sorted and unsorted data. Binary search repeatedly divides a sorted list into two halves. It compares the key with the middle element and continues in the half that may contain the key. Binary search requires the list to be sorted.

Algorithm

1. Read the customer IDs and the ID to be searched.

2. For linear search, start from the first element and compare each ID with the search key.

3. If a match is found, report its position; otherwise report that it is not present.

4. For binary search, first arrange the IDs in ascending order if they are not already sorted.

5. Set low to the first index and high to the last index.

6. While low is less than or equal to high, calculate mid = (low + high) // 2.

7. If the middle element equals the key, report success. If the key is smaller, set high = mid - 1; otherwise set low = mid + 1.

8. If the search interval becomes empty, report that the key is not present. Stop.

Time Complexity

Linear search: O(n) worst case. Binary search: O(log n) worst case on sorted data. Sorting first, if required, typically takes O(n log n) with an efficient sorting algorithm.

Space Complexity

O(1) auxiliary space for the iterative versions of both searches.

Conclusion

The practical demonstrates linear search and binary search on customer ids using Python and the relevant data structure or algorithm.

Practical 3

Selection Sort on Employee Salaries

Aim

To arrange employee salaries in ascending order using the Selection Sort technique.

Theory

Selection Sort is a comparison-based sorting algorithm. It divides the list into a sorted part and an unsorted part. In each pass, it finds the smallest element in the unsorted part and swaps it with the first unsorted element. After each pass, one more element is placed in its correct position. The method is simple and performs sorting in place.

Algorithm

1. Read the employee salaries into a list.
2. Set the current position i from 0 to n - 2.
3. Assume the element at i is the minimum and store its index.
4. Scan the remaining unsorted elements to find the smallest salary.
5. Swap the smallest salary with the element at position i.
6. Repeat until all positions have been processed.
7. Display the sorted salaries and stop.

Time Complexity

$O(n^2)$ in best, average, and worst cases because the remaining unsorted part is scanned on every pass.

Space Complexity

O(1) auxiliary space because sorting is performed in the original list.

Conclusion

The practical demonstrates selection sort on employee salaries using Python and the relevant data structure or algorithm.

Practical 4

Bubble Sort and Top Five Highest Salaries

Aim

To sort employee salaries using Bubble Sort and display the five highest salaries.

Theory

Bubble Sort repeatedly compares adjacent elements and swaps them when they are in the wrong order. After each pass, the largest remaining element moves to the end of the unsorted portion. Once the list is sorted in ascending order, the last five elements represent the five highest salaries. An optimized version can stop early if a pass makes no swaps.

Algorithm

1. Read all employee salaries into a list.
2. Set the number of passes to n - 1.
3. Compare each pair of adjacent elements in the unsorted portion.
4. If the left element is greater than the right element, swap them.
5. Repeat the passes until the list is sorted in ascending order; optionally stop if no swap occurs in a pass.
6. Display the sorted list.
7. Display the last five elements in reverse order to show the highest salary first. If fewer than five salaries exist, display all available salaries.
8. Stop.

Time Complexity

$O(n^2)$ average and worst case. With an early-stop check, the best case is O(n).

Space Complexity

O(1) auxiliary space for in-place Bubble Sort.

Conclusion

The practical demonstrates bubble sort and top five highest salaries using Python and the relevant data structure or algorithm.

Practical 5

Undo and Redo Operations Using Stacks

Aim

To implement Undo and Redo operations of a simple text editor using two stacks.

Theory

A stack follows the Last In, First Out (LIFO) principle. An undo stack stores previous document states, while a redo stack stores states that can be restored after an undo. Before making a change, the current document state is pushed onto the undo stack. When undo is performed, the current state is pushed onto the redo stack and the previous state is restored. A new edit clears the redo history because the earlier redo path is no longer valid.

Algorithm

1. Initialize an empty document, an undo stack, and a redo stack.

2. When a new change is entered, push the current document onto the undo stack, apply the change, and clear the redo stack.

3. For Undo, if the undo stack is not empty, push the current document onto the redo stack and restore the top state from the undo stack.

4. For Redo, if the redo stack is not empty, push the current document onto the undo stack and restore the top state from the redo stack.

5. If a requested stack is empty, display an appropriate message.

6. Display the document whenever required and repeat until Exit is selected.

7. Stop.

Time Complexity

Stack push and pop are O(1). Restoring or copying a complete document can take O(L), where L is the document length; appending text may also take O(L) depending on implementation.

Space Complexity

O(kL) in the snapshot approach, where k is the number of saved states and L is the average document length.

Conclusion

The practical demonstrates undo and redo operations using stacks using Python and the relevant data structure or algorithm.

Practical 6

Event Processing System Using Queue

Aim

To implement an event-processing system in Python using a queue, supporting adding, processing, displaying, and cancelling events.

Theory

A queue is a linear data structure that follows First In, First Out (FIFO). The first event added is the first event processed. New events are inserted at the rear, and processing removes an event from the front. A queue is suitable for event handling because it preserves arrival order. A list can demonstrate queue operations, although removing the first list element requires shifting the remaining elements.

Algorithm

1. Create an empty queue.
2. To add an event, read its details and insert it at the rear of the queue.
3. To process an event, check whether the queue is empty; if not, remove and display the front event.
4. To display events, traverse the queue from front to rear.
5. To cancel an event, search for the requested event and remove it if it exists.
6. Repeat the menu until Exit is selected, then stop.

Time Complexity

With a Python list, adding at the rear is O(1) amortized, processing from the front using pop(0) is O(n), displaying is O(n), and cancellation by value is O(n). A deque supports O(1) append and popleft.

Space Complexity

O(n) for storing n pending events.

Conclusion

The practical demonstrates event processing system using queue using Python and the relevant data structure or algorithm.

Practical 7

Call Center Call Handling Using Queue

Aim

To manage incoming customer calls in the order received using a queue containing customer ID and call time.

Theory

A call center queue follows the FIFO principle. Each call is represented by its customer ID and call time. New calls are added at the rear, and the call at the front is answered first. This ensures that calls are handled in their arrival order. The queue can also be displayed to show waiting calls and checked to determine whether it is empty.

Algorithm

1. Initialize an empty call queue.
2. When a call arrives, read the customer ID and call time and insert the pair at the rear.
3. When an agent is ready, check whether the queue is empty.
4. If the queue is not empty, remove the front call and display its customer ID and call time.
5. Display all waiting calls when requested.
6. Display an appropriate message if the queue is empty.
7. Repeat until Exit is selected, then stop.

Time Complexity

Using a Python list, insertion at the rear is O(1) amortized, answering with pop(0) is O(n), and displaying all calls is O(n). Using collections.deque makes insertion and removal at the ends O(1).

Space Complexity

O(n) for storing n waiting calls.

Conclusion

The practical demonstrates call center call handling using queue using Python and the relevant data structure or algorithm.

Practical 8

Student Record Management Using Singly Linked List

Aim

To create a student record management system using a singly linked list with operations to add, delete, update, search, sort, and display student records.

Theory

A singly linked list consists of nodes. Each node stores student information such as roll number, name, and marks, together with a link to the next node. Nodes are created dynamically and are not required to occupy consecutive memory locations. The list can grow or shrink during execution. Records can be searched or updated by traversing the links. Sorting can be performed by comparing roll numbers or marks and swapping the complete record fields together.

Algorithm

1. Define a node containing roll number, name, marks, and a next link. Initialize head to None.

2. To add a student, create a node and attach it at the end of the list.

3. To delete a student, search for the roll number, adjust the previous node's link (or head), and remove the node.

4. To update a student, search by roll number and replace the name and marks.

5. To search, traverse the list and compare each node's roll number with the required roll number.

6. To sort, compare pairs of nodes using roll number or marks and swap all student data fields together when their order is incorrect.

7. To display in ascending or descending order, traverse the sorted list and print each record.

8. Repeat the menu until Exit is selected.

Time Complexity

Add at end: O(n) without a tail pointer; delete, update, and search: O(n); display: O(n); simple pairwise sorting: $O(n^2)$.

Space Complexity

O(n) for n student nodes. The linked-list links require constant space per node.

Conclusion

The practical demonstrates student record management using singly linked list using Python and the relevant data structure or algorithm.

Practical 9

Expression Tree from Prefix Expression

Aim

To construct an expression tree from a prefix expression, perform non-recursive postorder traversal, and delete the complete tree.

Theory

An expression tree is a binary tree in which leaf nodes represent operands and internal nodes represent operators. A prefix expression places each operator before its operands. The tree can be constructed by scanning the prefix expression from right to left and using a stack of nodes. For an operator, two nodes are popped and attached as its left and right children. Non-recursive postorder traversal can be performed with two stacks. The tree can also be traversed iteratively to release all nodes.

Algorithm

1. Read a valid prefix expression and initialize an empty stack.

2. Scan the expression from right to left.

3. For an operand, create a node and push it onto the stack.

4. For an operator, pop one node as the left child and the next node as the right child; create the operator node, attach both children, and push the new node.

5. After scanning, the remaining stack node is the root of the expression tree.

6. For non-recursive postorder, push the root onto stack 1. Repeatedly pop a node from stack 1, push it onto stack 2, and push its children onto stack 1.

7. Pop and print all nodes from stack 2 to obtain Left-Right-Root order.

8. Use a stack to visit all nodes, save their child references, and clear or release each node. Set the root to None.

9. Stop.

Time Complexity

Tree construction: O(n); two-stack postorder traversal: O(n); deletion traversal: O(n), where n is the number of expression symbols/nodes.

Space Complexity

O(n) for the expression tree and auxiliary stacks in the worst case.

Conclusion

The practical demonstrates expression tree from prefix expression using Python and the relevant data structure or algorithm.

Note: For the example prefix expression +--a*bc/def, the postorder sequence is a b c * - d e f / - +.

Practical 10

Pizza Delivery Using Weighted Graph and Dijkstra's Algorithm

Aim

To represent nearby locations as a weighted graph and find the minimum travel time from the pizza shop to every customer location using Dijkstra's algorithm.

Theory

A weighted graph consists of vertices and edges. Each vertex represents a location, and each edge represents a route between two locations. The edge weight represents travel time. Dijkstra's algorithm finds the shortest distance from one source vertex to all reachable vertices when all edge weights are non-negative. A dictionary-based adjacency list can store each location and its neighbours with travel times. This solves the shortest path from the shop to each customer individually; it is not the same as finding one minimum-total-time tour that visits every customer exactly once.

Algorithm

1. Represent each location as a vertex and each route as an edge weighted by travel time.

2. Build an adjacency list for the weighted graph.

3. Set the source (pizza shop) distance to 0 and all other distances to infinity.

4. Select the unvisited vertex having the smallest known distance.

5. Mark that vertex as visited and examine each of its neighbours.

6. Calculate the alternative distance through the selected vertex. If it is smaller than the neighbour's current distance, update the neighbour's distance.

7. Repeat until all reachable vertices have been visited or no reachable unvisited vertex remains.

8. Display the minimum travel time from the pizza shop to each location.

Time Complexity

For the simple implementation that scans all vertices to select the next minimum, $O(V^2 + E)$, commonly written $O(V^2)$ for a simple adjacency-list implementation. A priority-queue implementation can achieve $O((V + E) \log V)$.

Space Complexity

$O(V + E)$ for the adjacency list, distance data, visited set, and supporting structures.

Conclusion

The practical demonstrates pizza delivery using weighted graph and dijkstra's algorithm using Python and the relevant data structure or algorithm.

Practical 11

City Management Using Binary Search Tree

Aim

To store city names and their populations in a Binary Search Tree and perform insertion, deletion, population update, searching, and ascending/descending display.

Theory

A Binary Search Tree (BST) is a binary tree in which keys smaller than a node are stored in its left subtree and larger keys are stored in its right subtree. In this application, the city name is the key and population is the associated value. Inorder traversal displays city names in ascending order, while reverse inorder traversal displays them in descending order. Searching, insertion, and deletion depend on the height of the tree. In an unbalanced BST, the height can become n.

Algorithm

1. Define a node containing city name, population, left link, and right link. Initialize the root to None.

2. To insert a city, compare its name with the current node and move left or right until an empty position is found. Store the population with the city.

3. To search or update, follow the comparisons from the root until the city is found or a None link is reached.

4. To delete a city, locate its node. If it has no child or one child, replace it with its child. If it has two children, copy its inorder successor's city and population, then delete the successor.

5. To display ascending order, perform inorder traversal: left, root, right.

6. To display descending order, perform reverse inorder traversal: right, root, left.

7. To find the maximum number of comparisons for a search in the current tree, determine the number of nodes on the longest root-to-node path (the tree height measured in nodes).

8. Repeat the menu until Exit is selected.

Time Complexity

Search, insertion, update, and deletion: O(h), where h is tree height; O(log n) average for a reasonably balanced tree and O(n) worst case. Display: O(n).

Space Complexity

O(n) for n tree nodes. Recursive traversal or operations may use O(h) call-stack space.

Conclusion

The practical demonstrates city management using binary search tree using Python and the relevant data structure or algorithm.

Practical 12

Binary Search Tree Operations

Aim

To implement the basic operations of a Binary Search Tree, including insertion, deletion, searching, and display.

Theory

A Binary Search Tree stores values in sorted structural order. For every node, values in the left subtree are smaller and values in the right subtree are larger. Insertion follows comparisons until an empty link is found. Searching follows the same comparison rule. Deletion has three cases: deleting a leaf, deleting a node with one child, and deleting a node with two children. Inorder traversal displays the values in ascending order.

Algorithm

1. Create a node structure with data, left child, and right child. Initialize root to None.

2. To insert a value, compare it with the current node and recursively move left or right until an empty position is found; insert the new node there.

3. To search, compare the key with the current node. If equal, report found; if smaller, search left; otherwise search right.

4. To delete, locate the required node.

5. If the node is a leaf, remove it. If it has one child, replace it with that child. If it has two children, replace its value with the minimum value from the right subtree and delete that successor.

6. To display the tree in ascending order, perform inorder traversal: left, root, right.

7. Repeat the menu until Exit is selected.

Time Complexity

Insertion, search, and deletion: O(h), where h is tree height; average O(log n) and worst O(n). Inorder display: O(n).

Space Complexity

O(n) for storing n nodes. Recursive operations use O(h) auxiliary call-stack space.

Conclusion

The practical demonstrates binary search tree operations using Python and the relevant data structure or algorithm.

Practical 13

Hash Table Using Division Method and Linear Probing

Aim

To design a fixed-size hash table using the division method for hashing and linear probing for collision resolution, with insertion, searching, deletion, and display operations.

Theory

A hash table stores keys in an array using a hash function. The division method calculates the initial index as key modulo table size: h(key) = key % size. A collision occurs when two keys map to the same index. Linear probing resolves a collision by checking consecutive positions using (h(key) + i) % size. During deletion, a special DELETED marker (tombstone) is used instead of an empty slot so that later keys in the same probe sequence remain searchable.

Algorithm

1. Create an array of fixed size and initialize every slot as empty.

2. To insert a key, calculate its initial index using key % size.

3. If the slot is occupied by another key, check the next slot using linear probing until an empty or DELETED slot is found.

4. Insert the key if a free slot is found; report table full if every slot is occupied. Avoid inserting duplicate keys.

5. To search, start at the hash index and follow the same probing sequence. Stop if the key is found or a truly empty slot is reached; continue past DELETED markers.

6. To delete, search for the key and replace its slot with a DELETED marker.

7. To display, print every index and its current value.

8. Repeat until Exit is selected.

Time Complexity

Average insertion, search, and deletion are O(1) when the load factor is low; worst case is O(n). Display is O(n).

Space Complexity

O(m) for a table of fixed size m.

Conclusion

The practical demonstrates hash table using division method and linear probing using Python and the relevant data structure or algorithm.

Practical 14

Graph Traversal Using BFS and DFS

Aim

To represent a graph using an adjacency matrix for DFS and an adjacency list for BFS, and display the traversal sequences starting from vertex A.

Theory

A graph consists of vertices and edges representing locations and routes. Breadth First Search (BFS) visits vertices level by level and uses a queue. Depth First Search (DFS) explores one path as far as possible before backtracking and can use a stack. An adjacency matrix stores whether an edge exists between every pair of vertices. An adjacency list stores the neighbours of each vertex. Traversal order depends on the order in which neighbours are considered.

Algorithm

1. Define the vertices and represent the graph using an adjacency matrix for DFS.

2. Initialize an empty visited list and a stack containing the starting vertex A.

3. For DFS, pop a vertex. If it is not visited, mark and display it. Push its unvisited adjacent vertices in reverse order if a particular left-to-right visit order is required.

4. Continue until the DFS stack is empty.

5. Represent the same graph using an adjacency list for BFS.

6. Initialize an empty visited list and a queue containing A; mark vertices when they are enqueued to avoid adding them repeatedly.

7. Remove the front vertex from the queue, display it, and enqueue each unvisited neighbour.

8. Continue until the BFS queue is empty, then display both traversal sequences.

Time Complexity

DFS using an adjacency matrix: $O(V^2)$. BFS using an adjacency list: $O(V + E)$.

Space Complexity

Adjacency matrix: $O(V^2)$. Adjacency list: $O(V + E)$. Traversal stacks/queues and visited data require $O(V)$ additional space.

Conclusion

The practical demonstrates graph traversal using bfs and dfs using Python and the relevant data structure or algorithm.